

Define Problem Statements

CrediShield AI — Credit Card Approval Prediction System

1. Background & Context

Financial institutions receive thousands of credit card applications daily. Each application must be evaluated for **creditworthiness** — determining whether the applicant is likely to repay debt responsibly or default on payments.

Current Manual Process:

- **Duration:** 7–14 business days per application
- **Process:** Credit analyst manually reviews application form, checks credit bureau reports, calculates financial ratios, consults senior officers for borderline cases
- **Workforce:** Dedicated teams of 5–20 credit analysts per branch
- **Cost:** Average cost per manual evaluation: ₹500–₹2,000

Challenges with Manual Evaluation:

1. **Slow Processing:** Creates customer dissatisfaction and application backlogs
2. **Inconsistency:** Different analysts may reach different conclusions for identical applicant profiles
3. **Expensive:** High labor costs for repetitive evaluation tasks
4. **Human Bias:** Unconscious discrimination based on gender, age, marital status, or occupation type
5. **Scalability Issues:** Cannot handle sudden surges in applications (e.g., festive season promotions)

2. Primary Problem Statement

"How can we build an automated, data-driven system that accurately predicts whether a credit card application should be approved or rejected, based on an applicant's socio-demographic profile and historical repayment behavior?"

3. Sub-Problems Identified

Sub-Problem 1: No Direct Labels Exist

Description: The raw datasets (`application_record.csv` and `credit_record.csv`) do not contain an explicit "Approved" or "Rejected" column. Labels must be derived from payment behavior status codes.

Solution Approach: Engineer binary target labels from `credit_record.csv`:

- **Approved (1):** Clients whose worst payment status is within 30 days overdue (codes: C, X, 0, 1)
- **Rejected (0):** Clients who were ever 60+ days overdue (codes: 2, 3, 4, 5)

Sub-Problem 2: Severe Class Imbalance

Description: Over 98% of clients in the dataset have safe repayment histories (Approved), while only ~1.5% exhibit delinquent behavior (Rejected). Standard classifiers will be biased toward the majority class.

Impact: A naive model predicting "Approved" for everyone achieves 98% accuracy but fails to catch any risky applicants — defeating the purpose.

Solution Approach:

- Use stratified train-test splits to preserve class ratios
- Evaluate using F1-Score (not just accuracy) to account for minority class performance
- Include Balanced Random Forest Classifier which handles imbalance natively
- Compare 6 different models to find the best balance

Sub-Problem 3: Missing Data

Description: The `OCCUPATION_TYPE` field has approximately 30% missing values across the application records.

Solution Approach: Fill missing occupation values with 'Unknown' rather than dropping rows (to preserve data volume) and let the model learn this as a valid category.

Sub-Problem 4: Feature Heterogeneity

Description: The dataset contains a mix of:

- **Categorical variables:** Gender, education level, marital status, housing type, occupation, income type (text/string values)
- **Numerical variables:** Income amount, age, number of children, family size, years employed (continuous/discrete numbers)

Challenge: ML models require uniform numerical input — mixing types causes errors.

Solution Approach:

- Apply `LabelEncoder` to transform each categorical variable into integer codes
- Apply `StandardScaler` to normalize numerical features to zero mean and unit variance
- Store fitted encoders and scalers as `serialized` objects for consistent inference

Sub-Problem 5: Income Outliers

Description: Some applicants report extreme income values (e.g., \$1,000,000+ annual income) which are either data entry errors or ultra-high-net-worth individuals who distort statistical distributions.

Impact: Outliers skew the StandardScaler's mean and variance calculations, affecting all numerical features.

Solution Approach: Apply Interquartile Range (IQR) filtering to remove income values beyond 1.5× the interquartile range.

Sub-Problem 6: Deployment Constraints

Description: The solution must be deployable on free-tier cloud platforms (budget constraint) while being responsive enough for real-time predictions.

- Constraints:**
- Vercel: 500MB bundle size limit (exceeded — rejected)
 - Render Free Tier: 512MB RAM, cold starts after 15 min inactivity
 - GitHub: 100MB file size limit
- Solution Approach:**
- Pin exact dependency versions to minimize installation size
 - Use Gunicorn 22.0.0 (compatible with Python 3.12)
 - Exclude training-only libraries (matplotlib, seaborn) from production requirements
 - Serialize pre-trained models (~150KB total) and bundle them in the repository

4. Scope Definition

- In Scope:**
- Binary classification: Approve (1) / Reject (0) credit card applications
 - Web-based prediction dashboard with form input
 - REST API endpoints for prediction and metrics retrieval
 - PDF assessment report generation
 - Model comparison (6 algorithms)
 - Interactive Chart.js analytics
 - Prediction history with localStorage persistence
 - Cloud deployment on Render.com
 - Light/Dark theme support

- Out of Scope:**
- Real-time transaction fraud monitoring
 - Multi-currency or multi-region support
 - Regulatory compliance documentation (GDPR, RBI guidelines)
 - User authentication and role-based access control
 - Database integration (using localStorage only)
 - Mobile native application
 - API rate limiting or security hardening

5. Expected Outcomes

Outcome	Target	Achieved
Model Accuracy	≥ 95%	☑ 98.29%
F1-Score	≥ 90%	☑ 99.14%
Prediction Response Time	< 500ms	☑ ~85ms
Number of ML Models Compared	≥ 4	☑ 6 models
Functional Web Dashboard	Yes	☑ Implemented
PDF Report Generation	Yes	☑ Implemented
Cloud Deployment	Yes	☑ Render.com

Project Name: CrediShield AI

Author: Dinesh (dineshTech-creator)

Date: July 2026